

Analytics BI

Dashboard Creation and Visualization Platform

Role 6: Schema Inference Engine

Technical Report — Sprint 2

Group 8

March 2026

1. Role Overview

Role 6 is the Schema Inference Engine — the brain of the data pipeline. It runs automatically after any file is uploaded and parsed. Its job is to look at the raw data that just landed in MongoDB and answer three questions:

- What are the columns, and what data type does each hold?
- Is each column a Dimension (a category to group by) or a Measure (a number to aggregate)?
- Are any columns in this new dataset related to columns in other datasets that already exist in the system?

The answers are written into a Metadata document in MongoDB, which the Visualization Wizard reads later to power the drag-and-drop chart builder. Without this engine, the wizard would have no idea what fields exist, what they contain, or how datasets connect to each other.

REQ-2.2: The system shall classify every data field as either a Dimension or a Measure. REQ-2.5: The system shall automatically link related data tables so the Visualization wizard has all the information it needs for complex charts.

2. Files in This Role

The engine is split across five files, each with a distinct responsibility:

- **inferSchema.js — the pipeline orchestrator (entry point)**
 - Coordinates the full 5-step process from sampling to saving.
- **classifyColumns.js — column classification engine**
 - Determines data type, dimension/measure role, and aggregation hints for every column.
- **relationshipMapper.js — cross-collection relationship detector**
 - Scores column pairs across multiple datasets and detects FK → PK links.
- **updateMetadata.js — MongoDB persistence layer**
 - Saves and upserts all inferred results into the Metadata collection.
- **Metadata.js — Mongoose schema definition**
 - Defines the shape of what gets stored: columns, relationships, and inference status.

3. The Full Pipeline — inferSchema.js

inferSchema.js is the entry point. It exports a single async function called runSchemaInference() which is called by the upload/parser role the moment data lands in MongoDB. The function runs five steps in sequence:

#	Function / Module	What Happens
---	-------------------	--------------

1	<code>sampleDocuments()</code>	Fetches up to 100 documents from MongoDB using <code>\$sample</code> aggregation. Falls back to a normal <code>find()</code> if <code>\$sample</code> fails.
2	<code>classifyAllColumns()</code>	Reads every column across all sampled documents, infers data type, detects role (dimension vs measure), and computes stats.
3	<code>getAllMetadata()</code>	Fetches the already-saved metadata for every other collection in the database.
4	<code>toRelationshipInput()</code>	Normalizes each existing metadata record into the standard shape expected by the relationship detector.
5	<code>detectRelationships()</code>	Scores every column pair across all collections and returns high-confidence relationships.
6	<code>saveMetadata()</code>	Upserts the complete result (columns + relationships) into the Metadata MongoDB collection.

3.1 Step 1 — Sampling Documents

Rather than scanning every single row (which could be millions), the engine samples up to 100 documents using MongoDB's `$sample` aggregation stage, which returns a random selection. This is fast and gives a statistically representative view of the data.

If `$sample` fails (some MongoDB environments don't support it well), the function falls back to a plain `find({}).limit(100)`. Either way, it guarantees a sample without crashing.

100 documents is a deliberate balance. Too few and edge cases are missed. Too many and the classification step gets slow. At 100 rows, classification typically completes in under 50ms even for wide datasets.

3.2 Step 2 — Column Classification

The sampled documents are passed to `classifyAllColumns()` from `classifyColumns.js`. This returns a rich descriptor for every column found in the sample. Full detail in Section 4.

3.3 Step 3 — Fetching Other Collections

The engine calls `getAllMetadata()` to retrieve saved metadata for every other collection already in the system. The current collection is excluded from this list. Each record is then normalized through `toRelationshipInput()` into a common format — handling the fact that older records may use `schema` while newer ones use `columns`, and that `rowCount` and `totalRows` may both exist.

3.4 Step 4 — Relationship Detection

The current collection (now with its classified columns) is combined with all other normalized collections and passed to `detectRelationships()`. This returns an array of high-confidence FK → PK relationships. Full detail in Section 5.

3.5 Step 5 — Saving Metadata

All results are passed to `saveMetadata()` which writes an upserted document into the Metadata collection. If any step throws, `markInferenceFailed()` is called to record the failure state so the UI can surface it to the user.

4. Column Classification — classifyColumns.js

This file is responsible for examining each column and returning a structured descriptor for it. It is the largest file in the role because most of the analytical intelligence lives here.

4.1 Name Normalization

Before any analysis begins, every column name is normalized. This is essential because real-world data files use wildly inconsistent naming — camelCase, snake_case, spaces, mixed caps.

The `normalizeColumnName()` function converts camelCase to snake_case first (inserting an underscore between lowercase and uppercase transitions), then replaces all non-alphanumeric characters with underscores, lowercases the whole string, and strips leading/trailing underscores.

For example: "CustomerID" becomes "customer_i_d", and "Total Revenue" becomes "total_revenue". This normalized form is then tokenized by splitting on underscores, giving a clean array of semantic tokens like ["total", "revenue"].

4.2 Keyword Heuristics

Two keyword lists drive much of the dimension/measure decision before any values are even examined:

- Dimension keywords: id, name, category, type, status, region, country, city, state, department, group, label, code, flag, gender, product, brand, segment, channel, year, month, quarter, date, and more.
- Measure keywords: revenue, sales, profit, amount, total, sum, count, quantity, price, cost, rate, ratio, percent, score, rating, salary, income, clicks, impressions, and more.

The tokens from the column name are compared against both lists. If more measure-keywords hit than dimension-keywords, the column leans toward being a measure. If the column name ends with a FK suffix (_id, _code, _key, _ref, _no, _num), it is immediately classified as a dimension regardless of anything else.

4.3 Data Type Inference

For each column, up to 50 sample values are inspected. Each value is classified as one of: number, boolean, date, or string. The dominant type — covering at least 80% of non-null values — becomes the column's data type. If no single type covers 80%, the column is labelled 'mixed'.

Data Type	Detection Rule
number	JavaScript typeof === "number", OR string matches numeric regex (e.g., "123", "12.5e3")
boolean	typeof === "boolean", OR string is "true"/"false"/"1"/"0"
date	instanceof Date, OR string passes Date.parse() AND contains a date separator (-, /, T, :) or month name

string	All other values that don't match any of the above
mixed	Multiple types detected in same column, with dominant type covering less than 80% of values
empty	All values in the sample are null, undefined, or empty string

The date detector deliberately ignores plain integers like '2024' to prevent year-as-integer columns from being misclassified as dates. It requires at least one date separator character or a month name to be present.

4.4 The Role Decision (Dimension vs Measure)

Once data type and keyword hits are known, a rule engine runs in priority order:

- Rule 1 — FK suffix: if the normalized name ends with `_id`, `_code`, `_key`, `_ref`, `_no`, or `_num`, the role is **dimension** with **97% confidence**. This is the strongest signal.
- Rule 2 — Non-numeric types: if the data type is string, boolean, or date, the role is **dimension** with **90% confidence**. You cannot sum a name or an average a date.
- Rule 3 — Numeric with measure keywords: if the type is number and measure keywords outnumber dimension keywords, the role is **measure** with **90% confidence**.
- Rule 4 — Numeric with low cardinality: if the type is number but has 12 or fewer unique values covering 20% or less of rows (e.g. a rating scale of 1–5), it is treated as a **dimension** (like an enum) with **78% confidence**.
- Rule 5 — Numeric default: if none of the above applies, the role is **measure** with **82% confidence**.
- Rule 6 — Mixed type: if the data type is mixed, whichever keyword list scored higher wins, at **70% confidence**.

4.5 Aggregation Hints

For measure columns, the engine suggests a default aggregation function. If the normalized column name contains 'rate' or 'percent', it suggests 'avg' (averaging a rate makes more sense than summing it). All other measures default to 'sum'. This hint is stored in metadata and used by the visualization wizard to pre-configure chart axes.

4.6 Statistics Per Column

Beyond role and type, each column descriptor also includes:

- `nullCount` — how many of the 50 sampled values were null, undefined, or empty. Used in the Data Reviewer UI to flag data quality issues.
- `uniqueCount` — how many distinct values appeared in the sample. Low `uniqueCount` on a numeric column signals an enum/category rather than a continuous measure.
- `sampleValues` — up to 5 distinct non-null example values, used for UI previews in the schema view.
- `confidence` — a number from 0.00 to 1.00 representing how certain the engine is about its role classification. Displayed to the user as a data quality indicator.

5. Relationship Detection — relationshipMapper.js

This is the most complex module. It compares every column in the newly uploaded collection against every column in every other known collection, scores each pair using up to four stacking signals, and returns only high-confidence relationships.

5.1 Why Relationship Detection Matters

When a user uploads both a 'customers' table and an 'orders' table, the visualization wizard needs to know that `orders.customer_id` points to `customers.customer_id`. Without this, the wizard cannot build charts that combine data from both tables (e.g., 'revenue by customer region'). The relationship mapper performs this detection automatically, without requiring the user to define joins manually.

5.2 Key Concepts: PK vs FK

Before scoring, the engine classifies each column's role within its own collection:

Column Name	Collection	Detected As
customer_id	customers	Primary Key — base 'customer' matches collection
customer_id	orders	Foreign Key — base 'customer' does not match 'order'
order_id	orders_2024	Primary Key — 'order' token found in collection tokens
revenue	sales	Not a key — no FK suffix, not processed by relationship mapper
id	any collection	Excluded — bare 'id' is too generic, causes false positives

The `isPrimaryKey()` function uses a multi-token approach: it extracts the base entity from the column name (e.g., `"order_id"` → base `"order"`), tokenizes the collection name, singularizes each token, and checks if the base tokens appear in the collection tokens. This correctly handles collection names like `"orders_2024"` by tokenizing to `["order", "2024"]`, singularizing to `["order", "2024"]`, and finding a match for base `"order"`.

5.3 The 4-Signal Scoring System

For every column pair (A from collection X, B from collection Y), the engine scores two directions: $A \rightarrow B$ and $B \rightarrow A$. The direction with the higher score is kept. Scores from all applicable signals are added together:

Signal	Score	What Triggers It
1. Exact name match	+0.40	Both columns share the same normalized name (e.g., <code>"customer_id"</code> in both collections)
2. FK pattern match	+0.50	Column has FK suffix (<code>_id</code> , <code>_code</code>) and its base entity matches the target collection name
3. Value overlap	+0.00 to +0.30	Actual sample values from both columns intersect (≥50% overlap needed)

4. High cardinality bonus	+0.05	Both columns have more than 10 unique values, suggesting they are meaningful keys
Type compatibility bonus	+0.05	Both columns share compatible data types (string ↔ number allowed for IDs)
Type incompatibility penalty	-0.20	Columns have fundamentally incompatible types (e.g., date vs boolean)
Target is primary key bonus	+0.10	The target column is detected as the primary key of its collection
Source is foreign key bonus	+0.05	The source column is detected as a foreign key in its own collection

Scores stack — there is no early exit. A column pair can accumulate score from all four signals simultaneously. The maximum possible raw score is approximately 1.45 (all signals at maximum), which is then capped at 1.00 when stored.

5.4 Early Filter and False Positive Guards

To keep performance reasonable, column pairs with zero join potential are skipped before scoring:

- If neither column has a FK suffix AND they do not share the same normalized name, the pair is skipped. This eliminates the vast majority of column combinations (e.g., 'revenue' vs 'region') instantly.
- Bare 'id' vs bare 'id' pairs are always skipped, regardless of any other signal. Sequential integers like [1,2,3] are always identical across unrelated tables and would create false positives with 100% value overlap.

The `columnPointsToCollection()` function uses token-exact matching (not substring matching) to prevent 'order' from incorrectly matching inside 'order_items'. The base entity of the FK column must exactly match the singularized tokens of the target collection name.

5.5 Threshold and Deduplication

After scoring, only relationships with a confidence score of 0.45 or higher are kept. This threshold was set so that a single strong signal (exact name match + type compatibility = 0.45) is sufficient to surface a relationship, while random column coincidences (which score near 0) are filtered out.

Duplicate directional pairs are collapsed by keeping the one with the highest confidence score. The final result is sorted descending by confidence, so the most certain relationships appear first.

6. Persistence Layer — updateMetadata.js

This file handles all MongoDB reads and writes for the inference engine. It uses Mongoose and exports four functions:

6.1 saveMetadata()

The primary write function. It takes a `collectionName` and all inferred data (columns, relationships, row count, filename, etc.) and performs a MongoDB upsert keyed on `datasetId`. If a metadata document already exists for this dataset (e.g., the user reuploaded the same file), it is updated in place rather than duplicated.

Before writing, the function runs two sanitizers:

`sanitizeColumns()` strips the 'confidence' field from each column. Confidence is used internally during scoring but is intentionally not stored — it is an implementation detail, not a user-facing property.

`sanitizeRelationships()` filters the full relationship list to only keep relationships where this collection is involved (either as the source or the target) and strips the internal 'strategy' and 'score' fields. This ensures that each dataset's metadata only records the relationships relevant to it.

Both the 'schema' field and the legacy 'columns' field are written with identical content, ensuring backward compatibility with any existing code that reads from either field name.

6.2 `markInferenceFailed()`

Called when any step in the pipeline throws an error. It upserts a minimal record with `inferenceStatus` set to 'failed' and the error message stored in `inferenceError`. This is what the Data Reviewer UI reads to show a warning to the user instead of an empty schema view.

6.3 `getAllMetadata()` and `getMetadataForCollection()`

Read-only helpers. `getAllMetadata()` fetches every metadata document (used in Step 3 of the pipeline to build the cross-collection comparison list). `getMetadataForCollection()` fetches a single document by collection name or `datasetId`, used by external APIs like the Data Review endpoint.

7. Database Schema — Metadata.js

`Metadata.js` defines the Mongoose model that represents a single dataset's metadata document in MongoDB. The document has three main sections:

7.1 Dataset-Level Fields

- `datasetId` — the unique identifier for this dataset, used as the upsert key. Indexed for fast lookup.
- `collectionName` — the MongoDB collection where the raw data lives.
- `fileName` / `sourceFileId` — traceability back to the original uploaded file.
- `uploadedBy` / `ingestionRule` — who uploaded it and whether it was a new, append, or replace operation.
- `totalRows` / `rowCount` — redundant fields kept in sync for backward compatibility with older pipeline code.

- `inferenceStatus` — one of 'pending', 'complete', or 'failed'. The UI polls this to know when inference is done.
- `inferenceError` — stores the error message if `inferenceStatus` is 'failed'.

7.2 ColumnSchema (per column)

Each entry in the `schema/columns` array follows the `ColumnSchema` sub-schema:

- `name` — the original column name as it appears in the data.
- `dataType` / `type` — the inferred data type. Both fields stored for compatibility.
- `role` — 'dimension' or 'measure'. Restricted by an enum to prevent invalid values.
- `suggestedAggregation` — one of 'sum', 'avg', 'count', 'min', 'max', or null. Used by the visualization wizard as a default.
- `sampleValues` — an array of mixed-type values. `Mongoose.Schema.Types.Mixed` allows any data type, equivalent to a `vector<any>` in typed languages.
- `nullCount` / `uniqueCount` — data quality and cardinality statistics.

7.3 RelationshipSchema (per relationship)

Each entry in the `relationships` array follows the `RelationshipSchema` sub-schema:

- `fromCollection` / `fromColumn` — the FK side of the relationship.
- `toCollection` / `toColumn` — the PK side of the relationship.
- `confidence` — a number between 0.0 and 1.0. The higher the score, the more signals agreed that this relationship exists.

The 'strategy' field (which records which signals fired, e.g. 'exact_name_match+fk_pattern') is deliberately NOT stored in the database. It is an internal debugging tool stripped out by `sanitizeRelationships()` in `updateMetadata.js`.

8. End-to-End Example

To make the full flow concrete, here is what happens when a user uploads a file called 'orders.csv' that contains the columns: `order_id`, `customer_id`, `product_id`, `total_amount`, `order_date`, and `status`. Assume a 'customers' collection already exists in the system with columns: `customer_id`, `name`, `region`, and `email`.

Step 1 — Sampling

The pipeline samples up to 100 documents from the new 'orders' collection.

Step 2 — Classification

`classifyAllColumns()` processes all 6 columns:

- order_id: FK suffix → dimension, confidence 0.97
- customer_id: FK suffix → dimension, confidence 0.97
- product_id: FK suffix → dimension, confidence 0.97
- total_amount: numeric type + measure keyword 'amount' → measure, suggested aggregation 'sum', confidence 0.90
- order_date: date type → dimension, confidence 0.90
- status: string type + dimension keyword 'status' → dimension, confidence 0.90

Step 3 — Fetch Other Collections

The metadata for the existing 'customers' collection is fetched and normalized.

Step 4 — Relationship Detection

detectRelationships() compares all column pairs between 'orders' and 'customers':

- orders.customer_id vs customers.customer_id: exact name match (+0.40) + fk_pattern (orders.customer_id is FK, customers.customer_id is PK, base 'customer' matches collection 'customers') (+0.50) + value overlap if sample IDs intersect (+up to 0.30) + type compat (+0.05) + PK bonus (+0.10) + FK bonus (+0.05) = up to 1.40, capped at 1.00. Relationship detected with very high confidence.
- All other pairs (e.g., orders.order_id vs customers.name) score 0 or below threshold and are discarded.

Step 5 — Save

saveMetadata() writes the full result for 'orders' including the detected relationship to 'customers'. The visualization wizard can now offer a join between these two collections automatically.

9. Bugs Fixed During Development

The following bugs were identified and fixed in the relationship mapper and persistence layer:

Bug 1 — sanitizeRelationships dropped valid relationships (updateMetadata.js)

The original filter kept only relationships where fromCollection matched the current collection name. This silently dropped every relationship where the new collection was on the PK (toCollection) side. Fixed by keeping relationships where the collection appears on either side.

Bug 2 — Early filter skipped same-name non-key columns (relationshipMapper.js)

The namesMatch variable was computed after the early-exit guard that used it, so columns like 'country' vs 'country' (which have matching names but no FK suffix) were skipped before scoring. Fixed by computing namesMatch before the guard.

Bug 3 — isPrimaryKey used a broken singularize function (relationshipMapper.js)

The original `singularize()` just stripped a trailing 's'. This caused 'orders_2024' to singularize to 'orders_202', so 'order_id' in 'orders_2024' was never recognized as a primary key and Signal 2 (worth +0.50) never fired. Fixed with a multi-token singularizer that handles the collection name token by token.

Bug 4 — Value overlap gated only on key-like columns (relationshipMapper.js)

Signal 3 (value overlap) only activated when at least one column had a FK/PK classification. This prevented shared dimension columns like 'country' or 'region' from gaining overlap score. Fixed by also activating Signal 3 when the column names match exactly.

Bug 5 — Threshold too high for single-signal relationships (relationshipMapper.js)

The 0.60 threshold required at least two strong signals. A solid exact-name-match relationship (score = 0.45) was always filtered out. Threshold lowered to 0.45 so that a single well-matched pair can surface.

10. Summary

The Schema Inference Engine (Role 6) delivers full automation of the two most technically demanding post-ingestion tasks: understanding what the data contains and understanding how datasets relate to each other. A developer picking up this codebase for the first time should trace the following path:

- Start at `inferSchema.js` — `runSchemaInference()` — this is the entry point and shows the full 5-step flow.
- Read `classifyColumns.js` — `classifyAllColumns()` and `classifyColumn()` — to understand how each column gets its type, role, and confidence.
- Read `relationshipMapper.js` — `detectRelationships()` and `scoreDirection()` — to understand how FK → PK links are discovered.
- Read `updateMetadata.js` — `saveMetadata()` and `sanitizeRelationships()` — to understand what gets written to MongoDB and what is intentionally stripped.
- Read `Metadata.js` to understand the complete shape of what is stored and what downstream consumers (the visualization wizard, the data review UI) can read.